Async Dart and agent-assisted coding

Futures, Streams, heavy computation, agent-assisted coding

Dinu-Ștefan Rusu ADMD

What you should leave with

Futures and async/await

Futures, async/await, and try/catch/finally around every network or database call.

Streams in the UI

FutureBuilder and StreamBuilder turn asynchronous values into widgets that rebuild on their own.

Heavy computation

Recognize CPU-bound code that freezes the UI, and move it off the main thread with compute().

Agent-assisted coding

Prompt it with real context, then review its output like any other pull request.

1

Asynchronous Dart

Futures, await, and error handling

Contents

		Page
1	**Asynchronous Dart**	3
2	Streams and heavy computation	10
3	Agent-assisted coding	21

The UI thread

- Every build, every layout pass and every gesture handler runs on the same thread as your Dart code.
- A function that takes `400` ms to return blocks all three for `400` ms. The screen stops responding.
- async and await exist so a function can wait for something without occupying that thread while it waits.
- The event loop and microtask queue that make this precise belong to Mobile and Embedded Computing (MEC), Lecture 2.

The thread blocks until your code yields control back to it.

Futures

```dart
Future<String> fetchName() {
  return Future.delayed(
    const Duration(seconds: 1),
    () => 'Dinu',
  );
}

void main() {
  fetchName().then(print);
  print('This runs first.');
}
```

Future<T> is a promise.
The value is not there yet, but the type of the eventual value is known now.
`then` runs a callback when the value arrives. `await` is sugar that reads like synchronous code: next slide.

async and await

```dart
Future<User> loadUser(String id) async {
  final r = await dio.get('/users/$id');
  return User.fromJson(r.data);
}

Future<void> refresh(String id) async {
  final user = await loadUser(id);
  setState(() => _user = user);
}
```

await reads top to bottom.
The compiler rewrites it into callbacks; you write it like synchronous code.
Each await is a suspension point: the function hands control back to the event loop and resumes when the Future completes.

Sequential and concurrent awaits

```dart
// Sequential: one after another
final user = await loadUser(id);
final posts = await loadPosts(id);

// Concurrent: both start now
final results = await Future.wait([
  loadUser(id),
  loadPosts(id),
]);
```

Sequential waits add up.
Two 200 ms calls in sequence cost about 400 ms.
Future.wait starts every Future immediately and completes when all of them do, or fails as soon as one does.

Errors in awaited calls

- A failure inside async code does not throw at the call site. It completes the Future with an error instead.
- await rethrows that error at the point of the await, so try, catch and finally behave as they do in synchronous code.
- Without a try around it, the error escapes as an unhandled async error. Users rarely report it; the screen just seems to do nothing.

Every await is a place your code can fail.

Handling errors around await

```
Future<void> refresh(String id) async {
  try {
    _user = await loadUser(id);
  } on DioException catch (e) {
    _error = 'Network error';
  } catch (e, stack) {   // anything else
    FirebaseCrashlytics.instance
        .recordError(e, stack);
  } finally {
    if (mounted) {
      setState(() => _loading = false);
    }
  }}
```

One specific catch, one general catch.
Handle the failure you expect by type, then log anything else. Check `mounted` before calling setState after an await. The widget may already be gone.

2 Streams and heavy computation

Streams, FutureBuilder, StreamBuilder, and compute()

Contents Page

1 Asynchronous Dart 3
2 Streams and heavy computation 10
3 Agent-assisted coding 21

Streams

```dart
// A Future is one value, later.
final name = await fetchName();

// A Stream is many values, over
// time.
Stream<int> ticker() async* {
  for (var i = 0; ; i++) {
    await Future.delayed(
      const Duration(seconds: 1),
    );
    yield i;
  }
}
```

await for reads a Stream.
Or subscribe with `.listen` for a callback style.
Streams are how WebSockets and sensor feeds reach your UI. Cancel every subscription you create by hand, in dispose().

Sources of Futures and Streams

- HTTP calls through dio or http return a Future. Lecture 7, networking.
- Future.delayed and Timer model waiting without a real request.
- sqflite and Drift queries return a Future; watching a table returns a Stream. Lecture 9, local storage.
- Firestore snapshots and WebSocket messages arrive as Streams. Lecture 8 and Lecture 11.

FutureBuilder

```
FutureBuilder<User>(
  future: _userFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    if (snapshot.hasError) return Text('Error: ${snapshot.error}');
    return UserCard(user: snapshot.data!);
  },
)
```

`connectionState` tells you where the Future is: waiting, then done. Check `hasError` before reading `data`.

Creating the Future outside build

Recreated every build

```
class ProfilePage extends
    StatelessWidget {
  Widget build(context) {
    return FutureBuilder(
      future: fetchUser(),
      builder: (c, s) => Body(s),
    );
  }
}
```

Stored once, in initState

```
class _ProfilePageState
    extends State<Page> {
  late final Future<User>
      _future = fetchUser();
  Widget build(context) =>
      FutureBuilder(
        future: _future,
        builder: (c, s) => Body(s),
      );
}
```

Store the Future once. A widget that calls fetchUser() inside build() refetches on every rebuild.

StreamBuilder

```
StreamBuilder<int>(
  stream: ticker(),
  initialData: 0,
  builder: (context, snapshot) {
    if (snapshot.hasError) {
      return Text('Error: ${snapshot.error}');
    }
    return Text('Tick: ${snapshot.data}');
  },
)
```

StreamBuilder subscribes when the widget builds and cancels when it is removed. You never call `listen` yourself. `initialData` covers the gap before the first event.

Combining a Future and a Stream

- Load what already exists with a Future: chat history, a cached list, the first page of results.
- Wrap that Future in a FutureBuilder so the screen shows a spinner, then the initial data.
- Subscribe to what changes with a Stream: new messages, a live price, a sensor reading.
- Nest a StreamBuilder inside, or beside, the FutureBuilder so new events update the same screen.

Most screens need both. A Future for what is already true, a Stream for what keeps changing.

FutureBuilder vs StreamBuilder

Aspect	FutureBuilder	StreamBuilder
Input	one Future	a Stream
Rebuilds	once, when it completes	on every new event
Cancels for you	nothing to cancel	yes, when the widget is removed
Snapshot carries	connectionState, data, error	connectionState, data, error
Typical source	an HTTP call, a database read	a WebSocket, a live query

Asynchrony and parallelism

```dart
// This function is async. It still freezes the UI.
Future<int> sumTo(int n) async {
  var total = 0;
  for (var i = 0; i < n; i++) {
    total += i;   // pure CPU work
  }
  return total;   // loop never yielded
}
```

async only helps when something else is doing the waiting: a socket, a disk, a timer.

Rule of thumb: waiting on I/O calls for await; burning CPU for longer than a frame calls for compute().

compute() for heavy work

```dart
// Must be a top-level or static function.
List<Photo> parsePhotos(String body) {
  final list = jsonDecode(body) as List;
  return list.map(Photo.fromJson).toList();
}

Future<List<Photo>> fetchPhotos() async {
  final res = await http.get(url);       // I/O
  return compute(parsePhotos, res.body);  // CPU
}
```

compute() spawns a short-lived isolate, runs one function and sends the result back. Arguments and results must be sendable: primitives, Strings, Lists and Maps.

Choosing between await and compute()

- Waiting on a network call, a disk read, a timer or a database calls for `await`. Something else is doing the work; your thread stays free to keep drawing frames.
- Burning CPU for longer than a frame calls for `compute()`. Nothing is waiting; the loop only gets a turn back when the function returns.

Spawning an isolate by hand with `Isolate.spawn`, the full event loop, and how Dart's isolates compare to a Go goroutine are MEC Lecture 2.

compute() is the escape you will reach for in this course. What is happening underneath it is next semester's material.

3

Agent-assisted coding

Tools, MCP, prompting, code review, lab rules

Contents

		Page
1	Asynchronous Dart	3
2	Streams and heavy computation	10
3	**Agent-assisted coding**	21

AI coding tools in 2026

Agentic CLIs

Claude Code, Codex CLI, Gemini CLI: they run in your repository, edit files and run your tests.

IDE assistants

Cursor, GitHub Copilot, JetBrains AI: completion plus in-editor chat.

Cloud agents

Long-running tasks against a checkout; what comes back is a pull request you review.

Review bots

Automated tools comment on your pull request before a human ever opens it.

Tool access (MCP)

Agents reach your files, shell, database and APIs through a tool protocol.

One layer down

All of them are a shell over a handful of frontier models; the shell is what differs.

Model Context Protocol (MCP)

- MCP, the Model Context Protocol, is how an agent calls real tools instead of only generating text.
- A typical agentic CLI can read and edit files, run your test suite, query a database, or call an internal API, all through the same protocol.
- That access is not free: a misread instruction can touch files or data you never intended to expose.

Tool access raises the stakes of a wrong prompt.

Agent strengths

- Boilerplate: scaffolding, data classes, test fixtures, glue code.
- Explaining unfamiliar code, stack traces and build errors.
- Mechanical refactors that touch many files at once.
- Writing tests.
- A first draft of a task you are unsure how to start.

Agent weaknesses

- Plausible but wrong APIs: Flutter’s surface moves fast.
- Security holes: hardcoded keys, unvalidated input, permissive database rules.
- Code that ignores conventions you never told it about.
- Silent drift away from the package versions you are on.
- The error paths and edge cases nobody asked for.

An agent produces confident output whether or not it is correct. Everything it writes still lands in the commit log under your name.

Common agent mistakes in Flutter

- An agent trained on older material may suggest `google_generative_ai` when `firebase_ai` is the current path. Lecture 12.
- It may reach for setState inside a class that is not a State, or skip const where it would help.
- It may hardcode an API key directly into a Dart file, which ships inside the compiled app. Lecture 7 and Lecture 10.
- It rarely writes the empty-list, no-network or expired-token branch unless you ask for it by name.

Read the diff as you would a stranger’s pull request.

Prompting with context

- Name the files, packages and versions the change has to fit into.
- Break multi-part work into numbered steps: one concern per prompt.
- State your conventions explicitly. It cannot guess your codebase.
- Ask for a diff or a test, not just a description.
- **Then read the diff yourself.**

```
// Weak
add pagination to the list

// Better
In feed_page.dart, convert ListView
to ListView.builder. Paginate: 20
items per page, fetch next page at
200px from bottom, show a spinner.

Conventions: Dio for HTTP, no
setState outside State, const
where possible. Add a widget test.
```

Reviewing agent-generated code

Read every line

If you cannot explain it in review, you cannot ship it, or defend it at the practical test.

Check the security surface

Hardcoded keys, unvalidated input, permissive database rules.

Test the edges

Empty list, no network, expired token: the paths nobody thought to ask for.

Check the API is current

Models learn from old docs. Verify the package and method still exist.

Keep the slices small

One concern per branch and pull request; generated code is only reviewable in slices.

You own it

“The agent wrote it” is not a defense in code review, and not one here either.

Rules for agent use in the labs

Rule	What it means in practice
Allowed	Use any agent or assistant you like, for every lab and the semester app.
Must be understood	You must be able to explain any generated line if asked, live, during the lab.
Must be reviewed	Read the diff before you commit it, exactly like a teammate’s pull request.

Summary

1. **await is not a thread.** It suspends one function and frees the event loop until the Future completes.
2. **A CPU-bound loop still blocks the UI.** Reach for compute() the moment work outlasts one frame.
3. **An agent's output is confident, not necessarily correct.** You review it, you understand it, you own it.

Further reading

dart.dev/language/async · FutureBuilder cookbook · code.claude.com/docs

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel